property using JavaDoc comments that can be converted into metadata, as I'll show you in a minute.

Listing 4.2 Defining custom properties in a Spring bean

```
package com.polarbookshop.catalogservice.config;

import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "polar")     ❶
public class PolarProperties {
  /**
   * A message to welcome users.
   */
  private String greeting;                     ❷

  public String getGreeting() {
    return greeting;
  }

  public void setGreeting(String greeting) {
    this.greeting = greeting;
  }
}
```

❶ Marks the class as a source for configuration properties starting with the prefix "polar"

❷ Field for the custom polar.greeting (prefix + field name) property, parsed as String

Optionally, you can add a new dependency on the Spring Boot Configuration Processor in your build.gradle file. That will automatically generate metadata for the new properties and store them in META-INF/spring-configuration-metadata.json when building the project. The IDE can pick them up, show you the description message for each property, and help you with autocompletion and type checks. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 4.3 Adding dependency for the Spring Boot Configuration Processor

```
configurations {          ❶
  compileOnly {
```